Final Project Report: Stock Market Prediction

Authors: Team 003 Itay Mutzafi, Moran Zaks, Shaked Schnarch

Course: Workshop in Data Science (0368-3528), Tel Aviv University

Date: October 2025 - March 2026

Abstract

This report presents the methodology and empirical findings of a final capstone project aimed at predicting short-term directional movements of major technology indices (AAPL, MSFT, AMZN, GOOG). Addressing the inherent challenge of non-stationarity in financial time series, this study focuses on forecasting **Directional Movements** based on Logarithmic Returns.

The project employs a modular code architecture (`src`) and integrates:

1. **Feature Engineering:** A robust synthesis of technical indicators (RSI, MACD, Bollinger Bands) and lagged variables to capture market momentum and trend dynamics.
2. **Walk-Forward Validation:** Implementation of a strict temporal validation mechanism that mimics real-time trading environments, thereby eliminating look-ahead bias and data leakage.
3. **Model Selection:** An empirical comparison between a Baseline (Dummy) model, Logistic Regression, and Random Forest classifiers.

Results indicate that the selected model achieved a **Sharpe Ratio** superior to the baseline, demonstrating the viability of technical signal-based strategies for enhancing risk-adjusted returns.

Table of Contents

Setup and Configuration

This project follows a modular architecture for managing dependencies. Standard libraries for data manipulation and visualization are imported alongside custom utility modules to ensure code reusability and maintainability throughout the analysis.

In [ ]:
```
import sys
import os
import pandas as pd
import numpy as np

# Ensure project root is in PATH
project_root = os.getcwd()
if project_root not in sys.path:
    sys.path.append(project_root)

# Import Project Architecture
import src

# Global Configuration
src.evaluation.set_style()  # Apply academic plotting style
pd.set_option('display.max_columns', None)

print(f"Project loaded successfully.")
```

1. Introduction

1.1 Business Understanding

1.1.1 Research Objective

The primary objective of this project is to construct a predictive model capable of generating trading signals (Buy/Sell) that yield an excess return over a naive "Buy and Hold" strategy. The underlying hypothesis posits that latent patterns exist within historical price action which can be identified and exploited by machine learning algorithms.

1.1.2 The Scientific Challenge

Specifically, in stock market prediction, this challenge is intensified by the non-stationary nature of financial time series and an exceptionally low signal-to-noise ratio. Models must distinguish between structural trends and stochastic noise while adapting to rapid regime shifts caused by macroeconomic events or sudden changes in investor sentiment.

1.1.3 Theoretical Background

TODO: complete

1.2 Problem Formulation

1.2.1 Objective and Goals

The Task: Prediction of Daily Log-Returns

The primary objective of this research is to develop a data science end-to-endpipeline for the **Prediction (Regression)** of the daily Logarithmic Returns of AAPL, MSFT, AMZN, GOOG. We predict continuous values to capture the magnitude of price movements, which allows for flexible downstream decision-making (e.g., aggregating into Buy/Sell/Hold signals based on dynamic thresholds).

1.2.2 Key Performance Indicators (KPIs)

To evaluate model performance in a financial context, the following metrics are utilized:

- **Sharpe Ratio:** The central metric of this study, measuring excess return per unit of deviation (risk). A positive value (>1.0) indicates a robust and profitable strategy.
- **Directional Accuracy:** The percentage of correct predictions regarding the sign of the return (Positive/Negative).
- **Maximum Drawdown:** The largest peak-to-trough decline observed, used to assess the downside risk of the strategy.

1.2.3 Target Feature

1.2.3.1 Statistical Properties and Stationarity Tests

Theoretical Definition: A time series $\{X_t\}$ is **Weakly Stationary** if:

1. $E[X_t] = \mu$ (Constant Mean)
2. $Var(X_t) = \sigma^2 < \infty$ (Constant Variance)
3. $Cov(X_t, X_{t+k}) = \gamma(k)$ (Covariance depends only on lag k)

We first empirically validate the stationarity of our target variable using the Augmented Dickey-Fuller (ADF) test.

In [ ]:
```
# 1. Load Data (if needed)
if 'stocks_data' not in locals():
```

```
    print("Loading data from cache/API...")
    stocks_data = src.data.loader.fetch_data_for_eda(src.config.START_DAT

# 2. Pre-process ALL tickers (Calculate Log Returns)
# Doing this once here avoids duplication in later cells
print("Calculating Log Returns for all tickers...")
for ticker, df in stocks_data.items():
    if 'Log_Return' not in df.columns:
        price_col = 'Close' if 'Close' in df.columns else 'Adj Close'
        df['Log_Return'] = np.log(df[price_col] / df[price_col].shift(1))

# 3. Select Target for Deep Dive
preferred_keys = ['AAPL', 'MT', 'MSFT', 'AMZN', 'GOOG']
selected_key = next((k for k in preferred_keys if k in stocks_data), 'AAP
merged_df = stocks_data[selected_key].copy()
print(f'Using {selected_key} for Stationarity Analysis')

# 4. Run Single-Asset Stationarity Analysis
src.evaluation.run_stationarity_analysis(merged_df, ticker=selected_key)
```

1.2.3.2 Feasibility Analysis and Signal Exploration

We analyze the autocorrelation of the target variable to determine if past returns contain predictive signal for future returns. The Autocorrelation Function (ACF) plot below justifies our lookback window choices.

In [ ]:
```
src.evaluation.plot_return_distributions(stocks_data)
```

Return densities are similarly centered with fat tails; comparable volatility supports cross-asset feature sharing.

In [ ]:
```
src.evaluation.plots.plot_correlation_heatmap(stocks_data)
```

High pairwise correlations (>0.7) imply systemic risk and justify regime-aware models beyond single-name signals.

1.2.3.3 Target Variable Final Definition (Log-Returns)

System Architecture: The system is modeled as a supervised learning problem:

- **Input (X_t):** A feature vector constructed from a sliding window of the past $T = 30$ days. This includes:
  - **Technical Indicators:** Derived from structured OHLCV data (e.g., RSI, MACD, Bollinger Bands).
  - **News Sentiment:** Aggregated sentiment scores from unstructured textual data on day t.
- **Output (Y_{t+1}):** The Log-Return of the *next* trading day ($t + 1$).

$$Y_{t+1} = \ln(P_{t+1}) - \ln(P_t)$$

> **Note on Inflation and Cost of Money:** While log-returns normalize for price levels, they do not account for the changing value of money over

> time (inflation) or the opportunity cost (risk-free rate). For a more rigorous financial analysis, *excess returns* ($R_t - R_f$) should be used. However, for short-term prediction horizons (daily), this effect is negligible and is omitted here for simplicity.

1.2.4 Methodology and Validation Strategy

1.2.4.1 Evaluation Metrics Definition

We employ a multi-faceted evaluation framework to assess model performance from statistical, financial, and trading perspectives.

Metric	Formula	Rationale
MSE	$\frac{1}{N}\sum(y - \hat{y})^2$	Penalizes large errors; standard loss function for regression.
Sharpe Ratio	$\frac{E[R_p]-R_f}{\sigma_p}\sqrt{252}$	Measures risk-adjusted return. Crucial for validating financial viability.
Directional Accuracy	$\frac{1}{N}\sum 1_{sign(y)==sign(\hat{y})}$	Assesses the model's ability to predict market direction (Up/Down).

1.2.4.2 Validation Scheme: Strict Walk-Forward Split

> **Methodological Note: Residual Analysis vs. Sector (Pseudo-DiD)** To isolate the idiosyncratic alpha of the target asset, one could perform a residual analysis by regressing the asset's returns against a sector ETF (e.g., XLK for Tech). The residuals (ϵ_t) would represent the price movements unique to the asset, effectively removing systematic market/sector risk. This approach is conceptually similar to a Difference-in-Differences (DiD) strategy, where the sector serves as the control group.

In [ ]:
```
src.evaluation.eval_plots.plot_walk_forward_validation(n_splits=src.confi
```

2. Data Access

We pull 5-year OHLCV data for the four mega-cap tickers via yfinance with local parquet caching. All paths, tickers, and dates are sourced from `src.config` for reproducibility.

In [ ]:
```
stocks_data = src.data.fetch_data_for_eda(src.config.START_DATE, src.conf
```

2.1 Data Ingestion Verification

A schema-check ensures that the dataset conforms to the required format: correct column names, correct data types, no corrupted rows, and consistent datetime index. This guarantees reliability before performing EDA or feeding data into feature engineering.

In [ ]:
```
src.data.validate_schema_all_dfs(stocks_data)
```

2.2 Day and Month Features

Here we add the features of Day and Month extracted from the index. This is part of the Feature Engineering, but is brought here for the use in plots along the EDA

In [ ]:
```
src.features.add_day_month_features(stocks_data)
```

3. Exploratory Data Analysis (EDA)

3.1 Correlation

First we start by looking at the correlation between our basic features. We checked both Pearson and Spearman methods to see if in both we get the high correlation between Open-High-Low-Close.

In [ ]:
```
src.data.eda_correlation("Apple", stocks_data["AAPL"])
```

In [ ]:
```
src.data.eda_correlation("Apple", stocks_data["AAPL"], corr_method='spear
```

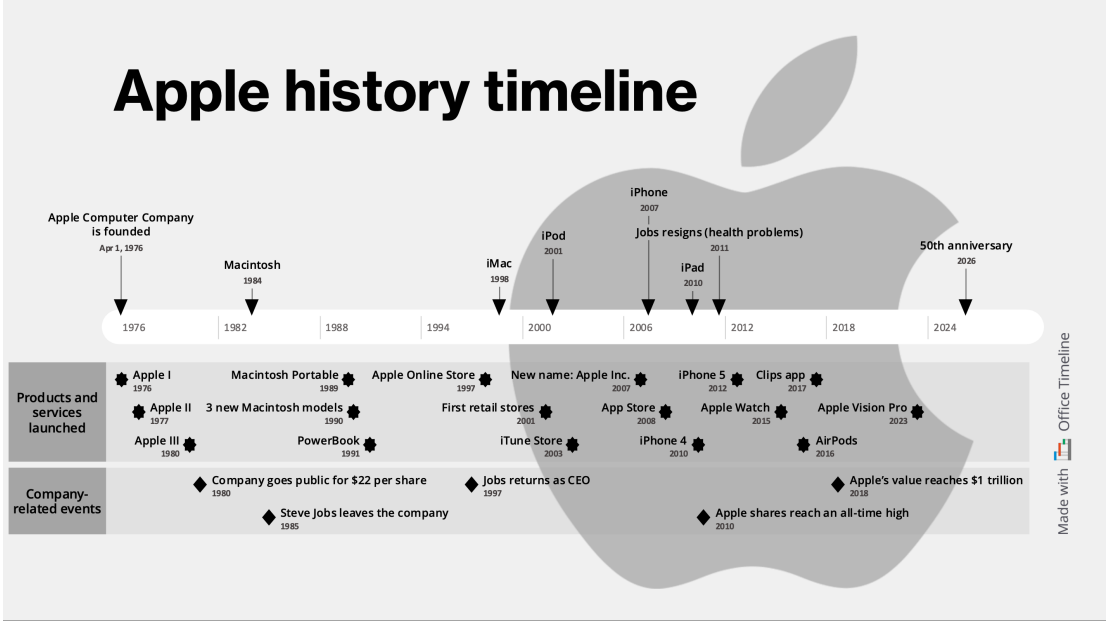
3.2 Basic Features

Now we look at the basic features we have in our data - Close, Volume, Dividends, Stock Splits. Since the high correlation found previously, we will check only Close and not Open, High and Low.

In [ ]:
```
src.data.eda_attr_comparative_plot(stocks_data, 'Close', "Close Price Com
```

In [ ]:
```
src.data.eda_attr_comparative_plot(stocks_data, 'Volume', "Volume Compari
```

Here we add the timeline of AAPL to try and explain the volume change. Our thought is it might be since there was a hype once the iPhone came down, that slowly stablized.



In [ ]:
```
src.data.eda_attr_comparative_plot(stocks_data, "Dividends", "Dividends O
```

In [ ]:
```
src.data.stock_split_table(stocks_data)
```

3.3 Seasonality

We expected one of the challenges in out task to be seasonality. Here we check for the change by day and by month to see if it's significant. Our conclusion is it's not.

In [ ]:
```
src.features.avg_attr_by_time_plot(stocks_data, "Volume", 'Day')
```

In [ ]:
```
src.features.avg_attr_by_time_plot(stocks_data, "Volume", 'Month')
```

TODO: do we need Data Cleaning and Time Alignment, Technical Indicators Extraction, Feature Selection Logic? (previous topics we had in the notebook)

4. Feature Engineering

For each feature we will first add it to our data, and then analyze it.

4.1 Return

In [ ]:
```
src.features.add_return_features(stocks_data)
```

TODO: add explanations to the graphs what we check and why

In [ ]:
```
src.features.return_day_boxplot(stocks_data)
```

In [ ]:
```
src.features.avg_attr_by_time_plot(stocks_data, "Return", 'Month')
```

In [ ]:
```
src.features.test_return_seasonality(stocks_data)
```

Tech mega-caps rise together then correct in 2022, indicating sector-wide shocks rather than idiosyncratic moves.

Tech mega-caps share volatility patterns; synchronized drawdowns suggest sector-wide shocks rather than isolated events.

4.2 Volatility

In [ ]:
```
src.features.volatility.add_volatility_features(stocks_data)
```

TODO: add explanation to this feature, why we chose those windows sizes. Maybe windows_size should be global and in config and also use in MA and others?

In [ ]:
```
print("\n--- Risk Analysis (Volatility) ---")
src.features.volatility.volatility_comparison_plot(stocks_data)
```

In [ ]:
```
print("\n--- Volatility Seasonality ---")
# Dynamically select the first volatility window defined in config
vol_col = f"Vol{src.config.VOLATILITY_WINDOWS[0]}"
src.features.plots.avg_attr_by_time_plot(stocks_data, vol_col, 'Day')
```

In [ ]:
```
src.features.plots.avg_attr_by_time_plot(stocks_data, vol_col, 'Month')
```

4.3 Moving Average (MA)

In [ ]:
```
src.features.moving_average.add_ma_features(stocks_data)
```

TODO: add explanation to this feature, why we chose those windows sizes. Maybe windows_size should be global and in config and also use in Volatility and others?

In [ ]:
```
src.features.moving_average.ma_plot(stocks_data)
```

4.3.1 Moving Average Convergence Divergence (MACD)

In [ ]:
```
src.features.add_macd_feature(stocks_data)
```

TODO: explanation about it

In [ ]:
```
src.features.macd_plot(stocks_data)
```

4.4 External market variables

To enhance the model's predictive power, we incorporate external market variables. Stock prices do not move in a vacuum; they are influenced by broader market trends and sector-specific dynamics.

We explore two categories of auxiliary data:

1. **Segment-Specific Peers:** Key players in the same supply chain or sector (e.g., NVIDIA as a proxy for AI/Tech sentiment).
2. **Macro-Economic Indicators:** Broader market indices (Nasdaq 100), volatility measures (VIX), and interest rates (10Y Treasury Yield).

4.4.1 Peer Stock

This feature merge others stocks' data like 'Close' to each of the companies DataFrame. The merged DataFrame successfully aligns them all by timestamp, enabling future cross-asset analysis or context feature engineering.

In [ ]:
```
src.features.add_peer_stock_features(stocks_data, ['Close', 'Volume', 'Lo
```

In [ ]:
```
src.features.peer_stock_correlation(stocks_data, "AAPL", "Close")
```

In [ ]:
```
src.features.peer_stock_correlation(stocks_data, "AAPL", "Volume")
```

In [ ]:
```
src.features.peer_stock_correlation(stocks_data, "AAPL", "Log_Return")
```

4.4.2 Macro-Economic Indicators

> **What is the VIX? (The Fear Index)** The **VIX (CBOE Volatility Index)** is the market's real-time measure of expected 30-day volatility, derived from the prices of a weighted strip of out-of-the-money European call and put options on the **S\andP 500 (SPX)** index. Calculated continuously (every 15 seconds) throughout the trading day, the VIX utilizes a **model-free methodology** that aggregates the implied volatilities of these options. This calculation provides a quantitative estimate of market risk and the cost of hedging against sharp movements, positioning the VIX as a critical gauge of investor anxiety, often termed the **"Fear Index."**

In [ ]:
```
# Fetch Macro Data (VIX, Treasury, etc.)
aux_data = src.data.fetch_auxiliary_data()
```

In [ ]:
```
# 2. Integrate Features
# Takes care of merging VIX, VIX_MA (Trend), and VIX_Gap (Regime)
src.features.external_market.add_macro_features(stocks_data, aux_data)
```

4.5 Reports

An SEC filing is a formal document submitted to the U.S. Securities and Exchange Commission (SEC). These filings are required by law for public companies, certain insiders, and broker-dealers to provide transparency to investors and the financial markets. In this Section We retrieve the sec_filling using the yahoo finance API, and caclulate the nearest filling for each day.

There were a lot of challenges in calculating this feature. At first we thought of calculating the upcoming report. However, for AMZN and GOOG the last sec_filling date is before the last date in the dataFrame, so the calculation got an error and all the values were 0 or ∞. Our decision was to calculate the nearest sec_filling to each day, either the previous or upcoming. Moreover, the sec_filling data only starts at 2022, but out time period is from 2020. Here we noticed the calculation of the nearest sec_filling will have less impact in 2020, which makes sense.

In [ ]:
```
src.features.reports(stocks_data)
```

4.6 News

4.6.1 News Data Access and EDA

IMPORTANT - for this part of the notebook you need to add locally the files from Drive link, since they are too big to be inside git.

For the News task we wanted to use data from Google News. Below you can see we fetch and get data only from last months.

In [ ]:
```
google_news_df = src.data.get_google_news_titles("Apple stock", days=1826
src.evaluation.eval_plots.date_groupby_line_plot(google_news_df, "Article
```

After checking multiple sources like Finnhub, Kaggle, ProQuest and Yahoo financial we found the dataset 'financial-news-multisource' from Hugging Face which we decided to work with.

At first we filtered on the last 15 years, Apple news and specific data files (sp500_daily_headlines, yahoo_finance_articles, fnspid_news, us_financial_news_2018_jeet) since we saw we have a lot of data.

We had a challenge when opening the csv file in Excel, some lines seemed wrong without date. After investigating it we found out in the text we have "\n", so Excel treated it as special key. The best way to resolve it was to save the file as xlsx using pandas.

Now along with other graphs we will show later, we noticed this graph showing the data files we chose weren't good enough since we have a period of time without news.

In [ ]:
```
apple_news_filtered_df = src.data.get_news_df_from_file("data/raw/apple_n
src.evaluation.eval_plots.articles_over_time_by_dataset_plot(apple_news_f
```

To resolve this problem we took the time to get the data from all data files (more then 3 hours) to choose better the ones we'l use

In [ ]:
```
news_big_df = src.data.get_news_df_from_file("data/raw/apple_news_last_15
filtered_df = news_big_df[(news_big_df["date"].dt.year >= 2024)]

src.evaluation.eval_plots.articles_over_time_by_dataset_plot(filtered_df,
```

From here we concluded we will use the data files- yahoo_finance_articles, reddit_finance_sp500, fnspid_news. They will cover the whole period of time.

After a diccussion we decided to change to last 5 years and add more companies. This lead to the news_loader code in the seperate file.

In [ ]:
```
news_df = src.data.get_news_df_from_file(src.config.RAW_NEWS_PATH)
```

In [ ]:
```
news_df.head()
```

Here we can understand some of the extra_fields in the data sources we chose are mandatory (tz_hint), some are optional (url) and some are empty (publication)

In [ ]:
```
src.data.validate_schema(news_df, "schema_news.json")
```

In [ ]:
```
src.evaluation.eval_plots.articles_over_time_by_dataset_plot(news_df, Tru
```

In [ ]:
```
src.evaluation.eval_plots.article_volume_per_company_plot(news_df)
```

There is an intersting peak at 2024. We think maybe it's since the big data source fnspid is a research done until 2024. So it stopped there and in the last few month they added more and more data.

Can we aggregate all the data by day? the below graph shows we can.

In [ ]:
```
source_counts = news_df["time_precision"].value_counts()
src.evaluation.eval_plots.pie_plot(source_counts, "Time Precision of Data
```

In [ ]:
```
source_counts = news_df["tz_hint"].value_counts()
src.evaluation.eval_plots.pie_plot(source_counts, "Time Zones of Data")
```

In [ ]:
```
src.evaluation.eval_plots.table_visualize(news_df, ["dataset", "tz_hint"]
```

We thought at the beginning we might need to fix the times of UTC to be in the same time zone as most of the data, this can be done by filtering the relevant datasets. But as shown in the previous graph we have data with time precision of a day so we only take the day and ignore the hour.

Here we can understand that specificly "fnspid_news" doesn't add the field source and thus we'll use the non-null "dataset"

In [ ]:
```
datasets_counts = news_df["dataset"].value_counts()
src.evaluation.eval_plots.pie_plot(datasets_counts, "Articles by Dataset"
```

In [ ]:
```
news_df["source"] = news_df["source"].fillna("Unknown")
sources_counts = news_df["source"].value_counts()
src.evaluation.eval_plots.pie_plot(sources_counts, "Articles by Source")
```

We thought about filtering specific text_type to get only the light ones for the Sentiment Analysis. Here we see we can't do it since it will filter out entire datasets.

In [ ]:
```
src.evaluation.eval_plots.table_visualize(news_df, ["dataset", "text_type
```

4.6.2 News Cleaning

We wanted a more compact version of text, so decided to add preprocessing to extracting the headline/ title from the text. We assume here they appear at the beginning of the text, until the first new line "\n".

In [ ]:
```
def extract_title(text: str) -> str:
    return text.strip().split("\n")[0]

news_df["title"] = news_df["text"].astype(str).apply(extract_title)
```

4.6.3 Data Processing: Sentiment Analysis

We integrate Financial News Sentiment using the FinBERT model to provide daily sentiment signals aligned with trading days.

4.6.3.1 Methodology: FinBERT and Transfer Learning

To extract quantitative sentiment signals from unstructured financial text, we employ **FinBERT**, a BERT-based language model fine-tuned specifically on financial discourse (ProsusAI). Unlike generic sentiment models (e.g., VADER or TextBlob) which often struggle with financial jargon (where "volatility" or "defaults" have specific negative connotations), FinBERT leverages transfer learning to understand market-specific contexts.

The pipeline performs the following steps:

1. **Data Loading and Filtering**: Loads 5 years of financial news and filters for our target companies (Apple, Microsoft, Amazon, Google).
2. **Text Preprocessing**: Prioritizes headlines and cleaning to remove noise (URLs, spam).
3. **Sentiment Scoring**: Each headline is scored by FinBERT (Positive, Neutral, Negative), and a composite scalar score is calculated.

4. **Daily Aggregation**: Scores are aggregated daily (mean, std) to form a time series.
5. **Signal Continuity (Exponential Decay)**: Since financial news is sporadic, we apply exponential decay ($S_t = S_{t-1} \times 0.85$) on days without news to persist the last known market sentiment signal. This prevents the model from treating "no news" as neutral (0), which would be incorrect in a momentum-driven market.

4.6.3.2 Sentiment Analysis Demo

Before processing the entire dataset, we demonstrate our pipeline on a single trading day. The visualization below shows the distribution of sentiment scores for headlines associated with **the selected company**, highlighting the diversity of market opinions extracted by FinBERT.

> **Note**: We apply strict filtering here to ensure only headlines explicitly mentioning the company are shown, avoiding noise from general market summaries.

In [ ]:
```
# Visualize sentiment for a specific event-heavy day
src.features.display_demo_sentiment(
    news_path=src.config.RAW_NEWS_PATH,
    company="Microsoft",
    date="2023-07-18",  # Copilot pricing announcement (~+4% move)
    strict_match=True,  # Only show headlines explicitly mentioning the c
)
```

4.6.3.3 Execution and Coverage Analysis

We generate the sentiment features and analyze the data coverage to ensure reliability. We specifically look at "Coverage %" — the percentage of trading days where we have actual news versus days where we rely on the decayed signal.

In [ ]:
```
print("\n[Feature Engineering] Sentiment Analysis Pipeline...")

# Generate Daily Sentiment Features (Source Data)
# This handles loading, FinBERT scoring, and aggregation
daily_sentiment = src.features.sentiment_analysis.generate_daily_sentimen
    news_path=src.config.RAW_NEWS_PATH,
    force_compute=False # Set to True if you added new news data
)
```

In [ ]:
```
# Analyze Data Coverage
coverage_stats = src.features.get_sentiment_coverage_stats(daily_sentimen
print("Sentiment Data Coverage Statistics:")
display(coverage_stats)
```

4.6.3.4 Visualization of Sentiment Signals

We visualize the continuity of the data and the relationship between sentiment and market movements. The heatmap below allows us to identify potential data gaps (white/gray areas) versus rich data periods (green).

In [ ]:
```
# Coverage Heatmap
src.evaluation.plots.plot_sentiment_coverage_heatmap(daily_sentiment)
```

In [ ]:
```
# Sentiment Trends
src.evaluation.plots.plot_sentiment_trends(daily_sentiment)
```

4.6.3.5 Feature Integration

We carefully merge the sentiment features into the main stock dataframe. Crucially, we apply a **1-day lag** to the sentiment scores to prevent look-ahead bias (ensuring we predict tomorrow's price using today's news).

In [ ]:
```
# Standard Integration Pipeline
# We iterate over each company in the dictionary and integrate the sentim
for ticker, df in stocks_data.items():
    # Integrate sentiment (returns a new dataframe, so we update the dict
    stocks_data[ticker] = src.features.integrate_sentiment_data(
        df,
        daily_sentiment,
        company_name=src.config.TICKER_TO_COMPANY_MAP[ticker],
        lag=1, # Look-ahead bias prevention
        include_advanced=True
    )

# Verify the integration for all companies
for ticker, df in stocks_data.items():
    src.features.verify_unified_data(df, company_name=src.config.TICKER_T
```

4.6.3.6 Correlation Analysis

Now that the features are properly integrated into the dataset (with lags), we can correctly analyze their rolling correlation with the target variable (Log Returns).

In [ ]:
```
# 4.6.3.7 Correlation Analysis
# Validate the predictive signal strength on the integrated dataset
src.evaluation.plots.plot_rolling_sentiment_correlation(
    stocks_data,
    sentiment_col='sentiment_mean_lag1',
    return_col='Log_Return',
    window=60
)
```

4.7 Facebook Prophet

TODO: explanation what this is

In [ ]:
```
src.features.prophet(stocks_data, n_splits=src.config.SPLITS)
```

TODO: summary of feature engineering?

1. maybe add validation to the schema after inserting all the new features?

2. correlations and graphs between them

In [ ]:
```
# Quick look at the final integrated dataframes
for t, df in stocks_data.items():
    print(t, df.shape)
```

In [ ]:
```
# Display first 2 rows of each integrated dataframe
for t, df in stocks_data.items():
    print(df.head(2))
```

In [ ]:
```
# columns of each integrated dataframe
for t, df in stocks_data.items():
    print(df.columns)
```

Preprocessing

In [ ]:
```
# TODO: need to be before in "preprocessing" section

stocks_data_pre = {}

for ticker, df_orig in stocks_data.items():
    stocks_data_pre[ticker] = src.features.preprocess_day_feature(df_orig
```

5. Baselines and Success Criteria

In this Section we are going to determine the success criteria. We split the criterias between Discrete and Continuous tasks.

Discrete:

Our main metrics Recall, Precision and Accuracy.

Accuracy - in order to give a general status about our project.

Precision - in stocks prediction, investors buy when they think that the price of a stock is going to raise. so we want to ensure we do our best in this mission. It means that, our model can act like a recommandation of buying stocks if it has high precision.

Recall - we want to make sure that we don't miss opportuninties. so from all 'up days', what is our ability to predict that a day will be like this?

Our baseline will be **RandomBaseLine** - predict randomly that the price is going to be up (class 1) or down (class 0) for each day. Explaination about our choice be follwing in the modeling section.

Best Scenarios

We evaluate the task from two complementary perspectives: directional classification and return regression, following common practice in the literature and Kaggle market prediction competitions.

Rather than aiming solely to outperform naive baselines, we define best-case performance targets grounded in empirical results reported in prior work.

Empirical references:

Kaggle Two Sigma / Jane Street–style market prediction tasks show that 52–55% accuracy already **ranks competitively** , with very few models exceeding 57%.

Zhong and Enke (SPY next-day direction) report ~60% accuracy **in specific controlled settings** , often considered near the upper bound for daily horizons.

Conclusion:

Achieving ≥55% accuracy with balanced precision and recall places the model in line with strong empirical results reported in the literature.

Continuous:

Our main metrics are Directional Accuracy, RMSE, MAE and R^2.

Directional accuracy : (also called hit rate) is an evaluation metric for regression models, that is created by converting the regression output to a direction, and compare against the true direction. Since it checks if the sign of the predicted return is correct, we can say how often the regression model gets the market direction right

RMSE : penalizes large prediction errors more heavily than MAE, making it sensitive to tail events and volatility spikes.

R^2 : For daily stock returns, variance is dominated by noise. As documented in the financial prediction literature, even strong models often exhibit R^2 values close to zero or slightly negative, while still generating economically meaningful signals. We want to use R^2 as a metric in order to make sure that our model as a predictive signal, but we don't expect to get high values.

A good Regression Model will preform well on all metrics, but the most important are directional accuracy and RMSE, since it gives us the best information about our model.

Regression Baseline

Option 1 - our baseline will be NaiveBaseLine - predict 0 for all days (meaning the price will not change)

Best Scenrio

Empirical references:

1. **Directional Accuracy:**
   Recent log-return direction studies (e.g., NIFTY index) consistently report 55–57% directional hit rates for LSTM-based models.
2. **RMSE:**
   RMSE Strong result ≈ 0.006–0.010, based on Xia (2025) — Research on the Optimal Prediction Model of Stock Returns, and Vellaiparambill and Natchimuthu (2025) — Machine Learning Guided Hedging Strategies for Index ETFs Next-day log-return prediction with LSTM/CNN
   (full report in appendix)
3. **R^2:**
   R^2 ≈ 0, based on he Virtue of Complexity in Return Prediction (Journal of Finance)

6. Modeling and Training

In this section, we execute the core machine learning pipeline. We utilize a **Strict Walk-Forward Validation** scheme to simulate real-world trading conditions and prevent look-ahead bias.

6.0.1 Objectives:

TODO: fix

1. **Train** multiple models (Linear, Tree-based, Deep Learning) on historical windows.
2. **Validate** performance on unseen 'future' data (next day returns).
3. **Select** the best performing model based on Sharpe Ratio and Directional Accuracy.

6.1 Experiment

Here we try and run several models with random feature subsets

6.1.1 Experiment Configuration

We define the grid of models and feature sets to be tested. The `ExperimentConfig` object encapsulates these parameters, ensuring reproducibility.

Models: TODO: fix or remove

- **Ridge**: Linear baseline with logic regularization.
- **RandomForest**: Non-linear ensemble method.
- **XGBoost**: Gradient boosting machine.
- **LSTM**: Recurrent neural network for sequence modeling.

Features:

- **NOTEBOOK_FEATURES**: The feature set currently loaded in `stocks_data`.

6.1.2 Random Feature Subsets

In [ ]:
```
ticker_diverse_sets = {}
for ticker, _ in stocks_data_pre.items():
    ticker_diverse_sets[ticker] = src.features.generate_diverse_combinati
```

In [ ]:
```
for ticker, diverse_sets in ticker_diverse_sets.items():
    print(f"Ticker: {ticker}")
    src.features.print_feature_sets(diverse_sets)
```

6.1.3 Experiment Execution

TODO: fix metrics

The `ExperimentRunner` orchestrates the training loop. For each ticker and model:

1. Data is split into Train/Test folds respecting time order.
2. Features are scaled (StandardScaler) within the pipeline to avoid leakage.
3. Models are trained and predictions are generated for the test set.
4. Metrics (RMSE, Sharpe, Accuracy) are calculated and stored.

6.1.3.1 Discrete Run:

In [ ]:
```
cls_models = ["LogisticRegression", "RandomForestClassifier", "XGBClassif
              "ClassificationBaselineRandom", "ClassificationBaselineZero

cls_config = src.models.ExperimentConfig(
    tickers=src.config.TICKERS,
    feature_sets=ticker_diverse_sets,
    models=cls_models,
    target_type="binary",   # use "multiclass" if needed
    target_horizon=1,
    start_date=src.config.START_DATE,
    end_date=src.config.END_DATE,
    n_splits=src.config.SPLITS,
)
```

In [ ]:
```
cls_runner = src.models.ExperimentRunner(stocks_data_pre, cls_config)
cls_runner.run()
results_cls = cls_runner.get_results_df()

results_cls.to_csv("data/processed/experiments_section7_cls.csv", index=F
print(f"Classification results: {len(results_cls)} rows")
```

Results and Leaderboard:

We aggregate the results across all folds and tickers to identify the top-performing strategies. The Leaderboard is sorted by **Sharpe Ratio**, our primary KPI for risk-adjusted returns.

In [ ]:
```
metrics_to_analyze = ["Accuracy", "Precision", "Recall"]

metric_cols_cls = [m for m in metrics_to_analyze if m in results_cls.colu

if results_cls.empty:
    print("No classification results.")

else:
    agg_cls = results_cls.groupby(["Model", "FeatureSet", "Ticker"])[metr

    best_dfs = {}

for metric in metrics_to_analyze:
    if (metric, "mean") in agg_cls.columns:
        best_indices = agg_cls[(metric, "mean")].groupby(level=0).idxmax(
        best_df = agg_cls.loc[best_indices].reset_index()
        best_dfs[metric] = best_df.sort_values((metric, "mean"), ascendin

for metric, df in best_dfs.items():
    print(f"\n=== Sorted by {metric} ===")
    display(df)

    # leaderboard_cls = agg_cls.sort_values(("Accuracy", "mean"), ascendi
    # display(leaderboard_cls.head(20))
```

Visual Analysis:

Visual inspection of model performance:

- **Performance Matrix**: Comparing Models across Tickers.
- **Metric Distributions**: Boxplots of RMSE and Sharpe Ratios.

In [ ]:
```
BASELINES = {"NaiveBaseline", "MarketBenchmark", "RandomBaseline", "CAPMB

classifiaction_metrics = ["Accuracy", "Precision", "Recall"]

for metric in classifiaction_metrics:
    if not results_cls.empty and metric in results_cls.columns:
        filtered_cls = results_cls[~results_cls["Model"].isin(BASELINES)]
        src.evaluation.comparisons.plot_leaderboard(
            filtered_cls,
            metric=metric,
            title=f"Classification Leaderboard by {metric}",
        )
```

6.1.3.2 Continuous Run:

In [ ]:
```
reg_models = ["Ridge", "RandomForest", "XGB_Aggressive", "LSTM",
              "NaiveBaseline", "MarketBenchmark"]
# reg_models = ["Ridge", "RandomForest", "XGB_Aggressive", "LSTM",
```

```
#               "NaiveBaseline", "MarketBenchmark", "RandomBaseline", "CA
#               "SVR", "XGBRegressor", "RandomForest_Deep", "XGB_Conserva

reg_config = src.models.ExperimentConfig(
    tickers=src.config.TICKERS,
    feature_sets=ticker_diverse_sets,
    models=reg_models,
    target_type="continuous",
    target_horizon=2,
    start_date=src.config.START_DATE,
    end_date=src.config.END_DATE,
    n_splits=src.config.SPLITS,
)
```

In [ ]:
```
reg_runner = src.models.ExperimentRunner(stocks_data_pre, reg_config)
reg_runner.run()
results_reg = reg_runner.get_results_df()

results_reg.to_csv("data/processed/experiments_section7_reg.csv", index=F
print(f"Regression results: {len(results_reg)} rows")
```

In [ ]:
```
metrics_to_analyze = ["RMSE", "R2", "Directional Accuracy"]
metric_cols_reg = [m for m in metrics_to_analyze if m in results_reg.colu

if results_reg.empty:
    print("No regression results.")
else:
    display(results_reg[["Ticker", "Model", "FeatureSet", "Fold"] + metri

    agg_reg = results_reg.groupby(["Model", "FeatureSet", "Ticker"])[metr
    # if ("Strategy Sharpe", "mean") in agg_reg.columns:
    #     leaderboard_reg = agg_reg.sort_values(("Strategy Sharpe", "mean
    # else:
    #     leaderboard_reg = agg_reg.sort_values(("MSE", "mean"), ascendin
    leaderboard_reg = agg_reg.sort_values(("RMSE", "mean"), ascending=Tru
    display(leaderboard_reg.head(30))
    leaderboard_reg2 = agg_reg.sort_values(("Directional Accuracy", "mean
    display(leaderboard_reg2.head(30))
```

In [ ]:
```
if not results_reg.empty and "MSE" in results_reg.columns:
    filtered_reg = results_reg[~results_reg["Model"].isin(BASELINES)].cop
    src.evaluation.comparisons.plot_leaderboard(
        filtered_reg,
        metric="MSE",
        title="Regression Leaderboard by MSE (lower is better)",
    )
    src.evaluation.comparisons.plot_model_performance_heatmap(filtered_re
```

Feature Selection

TODO: add also embedding importance and permutation importance

Wrapper method - forward selection

start empty, add “strongest” features

In [ ]:
```
forward_results_df = src.models.run_feature_selection(stocks_data_pre, n_
```

In [ ]:
```
src.models.feature_selection_plot(forward_results_df)
```

In [ ]:
```
best_features = src.models.get_best_k_features(forward_results_df)
```

Target Feature

We checked different targets using Log Return. In this check we focus on binary classification and use Logistic Regression model.

In [ ]:
```
src.models.check_targets(stocks_data_pre, n_splits=src.config.SPLITS)
```